

```
//SIT 55 : ASSIGNMENT 7 : GRAPH:MINIMUM SPANNING TREE
/*Represent a graph of your college campus using adjacency list
/adjacency matrix. Nodes should
represent the various departments/institutes and links should represent
the distance between them.
Find minimum spanning tree
a) Using Kruskal's algorithm.
b) Using Prim's algorithm.*/
```

```
#include <bits/stdc++.h>
using namespace std;
#define V 5
class PrimsAlgo
{
public:
int minKey(int key[], bool mstSet[])
{
int min = INT_MAX, min_index;
for (int v = 0; v < V; v++)
{
if (mstSet[v] == false && key[v] < min)
{
min = key[v], min_index = v;
}
}
return min_index;
}
void printMST(int parent[], int graph[V][V])
{
cout << "Edge \tWeight\n";
for (int i = 1; i < V; i++)
{
cout << parent[i] << " - " << i << " \t"<< graph[i][parent[i]] << " \n";
}
}
void primMST(int graph[V][V])
{
int parent[V];
int key[V];
bool mstSet[V];
for (int i = 0; i < V; i++)
{
key[i] = INT_MAX, mstSet[i] = false;
}
key[0] = 0;
parent[0] = -1;
for (int count = 0; count < V - 1; count++)
{
int u = minKey(key, mstSet);
mstSet[u] = true;
for (int v = 0; v < V; v++)
{
if (graph[u][v] && mstSet[v] == false && graph[u][v] <
key[v])
{
parent[v] = u, key[v] = graph[u][v];
}
}
}
}
```

```

printMST(parent, graph);
}
};
//-----
class Graph
{
    char vertices[10][10];
    int cost[10][10],no;
public:
    Graph();
    void creat_graph();
    void display();
    int Position(char[]);
    void kruskal_algo();
};
/* Initialzing adj matrix with 999 */
/* 999 denotes infinite distance */
Graph::Graph()
{
    no=0;
    for(int i=0;i<10;i++)
        for(int j=0;j<10;j++)
        {
            cost[i][j]=999;
        }
}
/* Taking inputs for creating graph */
void Graph::creat_graph()
{
    char Start[10],End[10];
    bool ac=true;
    int wt,i,j;
    cout<<"Enter the number of vertices: ";
    cin>>no;
    cout<<"\nEnter the vertices: ";
    for(i=0;i<no;i++)
        cin>>vertices[i];
    do
    {
        cout<<"\nEnter Start and End vertex of the edge: ";
        cin>>Start>>End;
        cout<<"Enter weight: ";
        cin>>wt;
        i=Position(Start);
        j=Position(End);
        cost[i][j]=cost[j][i]=wt;
        cout<<"\nDo you want to add more edges (1/0)? : "; /* Type 'Y' or 'y'
for YES and 'N' or 'n' for NO
*/
        cin>>ac;
    }while(ac=='y' || ac=='Y');
}
/* Displaying Cost matrix */
void Graph::display()
{
    int i,j;
    cout<<"\n\nCost matrix: ";
    for(i=0;i<no;i++)
    {
        cout<<"\n";

```

```

    for(j=0;j<no;j++)
    cout<<"\t"<<cost[i][j];
    }
}
/* Retrieving position of vertices in 'vertices' array */
int Graph::Position(char key[10])
{
    int i;
    for(i=0;i<10;i++)
    if(strcmp(vertices[i],key)==0)
    return i;
return -1;
}
void Graph::kruskal_algo()
{
    int i,j,v[10]={0},x,y,Total_cost=0,min,gr=1,flag=0,temp,d;
    while(flag==0)
    {
        min=999;
        for(i=0;i<no;i++)
        {
            for(j=0;j<no;j++)
            {
                if(cost[i][j]<min)
                {
                    min=cost[i][j];
                    x=i;
                    y=j;
                }
            }
        }

        if(v[x]==0 && v[y]==0)
        {
            v[x]=v[y]=gr;
            gr++;
        }
        else if(v[x]!=0 && v[y]==0)
            v[y]=v[x];
        else if(v[x]==0 && v[y]!=0)
            v[x]=v[y];
        else
        {
            if(v[x]!=v[y])
            {
                d=v[x];
                for(i=0;i<no;i++)
                {
                    if(v[i]==d)
                    v[i]=v[y];
                }//end for
            }
        }

        cost[x][y]=cost[y][x]=999;
        Total_cost=Total_cost+min; /* calculating cost of minimum spanning tree
*/
        cout<<"\n\t"<<vertices[x]<<"\t\t"<<vertices[y]<<"\t\t"<<min;

        temp=v[0]; flag=1;
    }
}

```

```

for(i=0;i<no;i++)
{
if(temp!=v[i])
{
flag=0;
break;
}
}
}
cout<<"\nTotal cost of the tree= "<<Total_cost;
}
//-----
int main()
{
bool a=true;
int ch;
Graph g;
PrimsAlgo pm;
while(a){
cout<<"Menu:\n 1. Prims algorithm \n 2. Kruskals algorithm \n";
cin>>ch;
switch(ch){
case 1:
{
int graph[V][V] = { { 0, 2, 0, 6, 0 }, { 2, 0, 3, 8, 5 }, { 0, 3, 0, 0, 7 }, { 6, 8, 0, 0, 9 }, { 0, 5, 7, 9, 0 } };
pm.primMST(graph);
}
break;
//-----
case 2:
{
g.creat_graph();
g.display();
cout<<"\n\n\nMinimum Spanning tree using kruskal algo=>";
cout<<"\nSource vertex\tDestination vertex\tWeight\n";
g.kruskal_algo();
}
break;
}
cout<<"\nDo you want to continue Program: (0/1) :";
cin>>a;
}
return 0;
}

```

```

/*OUTPUT:
Menu:
1. Prims algorithm
2. Kruskals algorithm
1
Edge Weight
0 - 1 2
1 - 2 3
0 - 3 6
1 - 4 5
Do you want to continue Program: (0/1) :1
Menu:
1. Prims algorithm

```

2. Kruskals algorithm

2

Enter the number of vertices: 5

Enter the vertices: 0 1 2 3 4

Enter Start and End vertex of the edge: 0

1

Enter weight: 2

Do you want to add more edges (Y=YES/N=NO)? : y

Enter Start and End vertex of the edge: 1

2

Enter weight: 3

Do you want to add more edges (Y=YES/N=NO)? : y

Enter Start and End vertex of the edge: 1

4

Enter weight: 5

Do you want to add more edges (Y=YES/N=NO)? : y

Enter Start and End vertex of the edge: 2

4

Enter weight: 7

Do you want to add more edges (Y=YES/N=NO)? : y

Enter Start and End vertex of the edge: 0

3

Enter weight: 6

Do you want to add more edges (Y=YES/N=NO)? : y

Enter Start and End vertex of the edge: 1

3

Enter weight: 8

Do you want to add more edges (Y=YES/N=NO)? : y

Enter Start and End vertex of the edge: 3

4

Enter weight: 9

Do you want to add more edges (Y=YES/N=NO)? : n

Cost matrix:

999 2 999 6 999

2 999 3 8 5

999 3 999 999 7

6 8 999 999 9

999 5 7 9 999

Minimum Spanning tree using kruskal algo=>

Source vertex Destination vertex Weight

0 1 2

1 2 3

1 4 5

0 3 6

Total cost of the tree= 16*/